

Ejercicio 1

Clase genérica

```
public class Set<T> {  
  
    private static final int TAM = 10;  
  
    private T[] array = (T[]) new Object[TAM];  
  
    // Insertar elemento  
  
    public boolean insert(T element) {  
        for (int i = 0; i < TAM; i++) {  
            if (array[i] == null) {  
                array[i] = element;  
                return true;  
            }  
        }  
  
        return false;  
    }  
  
    // Eliminar elemento  
  
    public boolean delete(T element) {  
        for (int i = 0; i < TAM; i++) {  
            if (array[i] != null && array[i].equals(element)) {  
                array[i] = null;  
                return true;  
            }  
        }  
  
        return false;  
    }  
  
    // Buscar elemento  
  
    public T find(T element) {  
        for (int i = 0; i < TAM; i++) {  
            if (array[i] != null && array[i].equals(element)) {
```

```

        return array[i];
    }
}
return null;
}
// Mostrar contenido
public void mostrar() {
    for (T elemento : array) {
        System.out.println(elemento);
    }
}
}

```

Main de prueba

```

public class Main {
    public static void main(String[] args) {
        // Set de Strings
        Set<String> nombres = new Set<>();
        nombres.insert("Adrian");
        nombres.insert("Mario");
        nombres.insert("Lucia");
        System.out.println("Buscar Mario:");
        System.out.println(nombres.find("Mario"));
        nombres.delete("Mario");
        System.out.println("Después de borrar:");
        nombres.mostrar();
        // Set de Integer
        Set<Integer> numeros = new Set<>();
        numeros.insert(10);
        numeros.insert(20);
    }
}

```

```

        numeros.insert(30);

        System.out.println("Buscar 20:");

        System.out.println(numeros.find(20));

        numeros.delete(20);

        System.out.println("Después de borrar:");

        numeros.mostrar();
    }
}

```

¿Puedes insertar una cadena en un Set<Integer>?

No.

Si intentamos hacer esto:

```

Set<Integer> numeros = new Set<>();

numeros.insert("Hola");

```

El programa dará error de compilación porque el conjunto está declarado para almacenar únicamente objetos Integer. Los genéricos garantizan seguridad de tipos y evitan errores de conversión.

Ejercicio 2

Clase excepción

```

public class SetOverflow extends Exception {

    public SetOverflow(String mensaje) {

        super(mensaje);

    }

}

```

Modificación del método insert

```

public boolean insert(T element) throws SetOverflow {

    for (int i = 0; i < TAM; i++) {

        if (array[i] == null) {

            array[i] = element;

            return true;

        }

    }

}

```

```

    }

    throw new SetOverflow("El conjunto está lleno");
}

```

Código de prueba

```

public class Main {

    public static void main(String[] args) {

        Set<Integer> numeros = new Set<>();

        try {

            for (int i = 0; i < 11; i++) {

                numeros.insert(i);

            }

        } catch (SetOverflow e) {

            System.out.println("ERROR:");

            System.out.println(e.getMessage());

        }

    }

}

```

Ejercicio 3

Método equals en la clase Set

```

@Override

public boolean equals(Object obj) {

    if (this == obj) {

        return true;

    }

    if (obj == null || getClass() != obj.getClass()) {

        return false;

    }

    Set<?> otro = (Set<?>) obj;

    for (int i = 0; i < TAM; i++) {

```

```

    if (array[i] == null && otro.array[i] != null) {
        return false;
    }
    if (array[i] != null && !array[i].equals(otro.array[i])) {
        return false;
    }
}
return true;
}

```

Código de prueba

```

public class Main {
    public static void main(String[] args) {
        Set<String> set1 = new Set<>();
        Set<String> set2 = new Set<>();
        set1.insert("A");
        set1.insert("B");
        set2.insert("A");
        set2.insert("B");
        System.out.println(set1.equals(set2));
    }
}

```

¿Puedes usar la instancia de T en el método equals? ¿Por qué?

No directamente.

En Java, los genéricos utilizan “type erasure” (borrado de tipos), lo que significa que en tiempo de ejecución el tipo T desaparece. Por eso no podemos comprobar directamente el tipo exacto de T con instanceof o comparaciones similares.

Ejercicio 4

Clase Equipo

```
public class Equipo<T> implements Comparable<Equipo<T>> {  
    private String nombre;  
    private int puntos;  
    public Equipo(String nombre, int puntos) {  
        this.nombre = nombre;  
        this.puntos = puntos;  
    }  
    public String getNombre() {  
        return nombre;  
    }  
    public int getPuntos() {  
        return puntos;  
    }  
    @Override  
    public int compareTo(Equipo<T> otro) {  
        return Integer.compare(otro.puntos, this.puntos);  
    }  
    @Override  
    public String toString() {  
        return nombre + " - " + puntos + " puntos";  
    }  
}
```

Clase Liga

```
import java.util.ArrayList;  
import java.util.Collections;  
public class Liga<T> {  
    private String nombre;
```

```

private ArrayList<Equipo<T>> equipos;

public Liga(String nombre) {
    this.nombre = nombre;
    equipos = new ArrayList<>();
}

public boolean agregarEquipo(Equipo<T> equipo) {
    if (equipos.contains(equipo)) {
        return false;
    }
    equipos.add(equipo);
    return true;
}

public void mostrarClasificacion() {
    Collections.sort(equipos);
    System.out.println("Liga: " + nombre);
    for (Equipo<T> e : equipos) {
        System.out.println(e);
    }
}
}

```

Jugadores

```

class Futbolista {}
class Baloncestista {}

```

Código de prueba

```

public class Main {
    public static void main(String[] args) {
        Liga<Futbolista> ligaFutbol = new Liga<>("La Liga");
        Equipo<Futbolista> madrid =
            new Equipo<>("Real Madrid", 80);
        Equipo<Futbolista> barcelona =

```

```

        new Equipo<>("Barcelona", 75);
ligaFutbol.agregarEquipo(madrid);
ligaFutbol.agregarEquipo(barcelona);
ligaFutbol.mostrarClasificacion();
// Esto da error de compilación
// Equipo<Baloncestista> lakers =
//     new Equipo<>("Lakers", 90);
// ligaFutbol.agregarEquipo(lakers);
    }
}

```

Explicación

El programa falla al compilar porque la liga está declarada como Liga<Futbolista> y solo acepta equipos de futbolistas. Java detecta automáticamente que Equipo<Baloncestista> es incompatible.

Ejercicio 5

```

public class Set<T extends Comparable<T>> {
    private static final int TAM = 10;
    private T[] array = (T[]) new Comparable[TAM];
    public boolean insert(T element) {
        // Evitar duplicados
        if (find(element) != null) {
            return false;
        }
        int posicion = 0;
        while (posicion < TAM &&
            array[posicion] != null &&
            array[posicion].compareTo(element) < 0) {
            posicion++;
        }
        if (posicion >= TAM || array[TAM - 1] != null) {

```



```
        return false;
    }
    for (int i = TAM - 1; i > posicion; i--) {
        array[i] = array[i - 1];
    }
    array[posicion] = element;
    return true;
}

public T find(T element) {
    for (T e : array) {
        if (e != null && e.equals(element)) {
            return e;
        }
    }
    return null;
}
}
```